



StackSniff

Status: 🚧 In Development

What is it?

StackSniff is a framework-aware path enumeration tool built for penetration testing and web application security assessments. Instead of brute-forcing a generic wordlist against every target, StackSniff first identifies the backend technology stack (e.g. Django, Laravel, WordPress) a web application is built on, then checks only the default paths, files, and endpoints known to be relevant to that specific stack.

Why build this?

Tools like Gobuster and ffuf are excellent at directory brute-forcing, but they're stack-agnostic — they don't know that a Laravel app is likely to expose `.env` or `storage/logs/laravel.log`, or that a Django app might leak details through a misconfigured debug page. Pentesters usually end up manually recalling or looking up these framework-specific paths during an engagement.

StackSniff automates that step: identify the stack, then go straight to the paths that matter for it — reducing noise and saving recon time during the reconnaissance phase of a pentest.

How it works (planned)

1. **Fingerprinting** — Detects the backend framework/CMS by inspecting HTTP response headers, cookies, and other signals (e.g. `X-Powered-By`, `laravel_session` cookie, Django's default error pages).
2. **Path enumeration** — Loads a curated list of default/sensitive paths specific to the detected stack (e.g. `.env`, `artisan`, `wp-config.php.bak`, `/admin/`) and checks which ones are accessible.
3. **Reporting** — Returns results with status codes and confidence indicators, flagging anything worth manual follow-up.

Planned features

- Framework/CMS fingerprinting (Laravel, Django, WordPress, and more over time)
- Curated, stack-specific default path wordlists
- False-positive handling for servers that return soft-404s
- Concurrent/threaded scanning for speed
- CLI with JSON output for chaining into other tools

Use case

Built primarily for use in authorized web application penetration testing engagements, as a faster, more targeted alternative/complement to generic directory brute-forcing during the reconnaissance phase.

This project is a work in progress, built as part of ongoing penetration testing skill development.